Business Impact: Results for the Presentation

Jake Hoopes

- 1. Default rate by model risk decile
- 2. Flagging the riskiest applicants
- 3. Same approval rate: the model vs. “no credit history, no loan”
- 4. Summary for the slides
- 5. Limitations

Purpose. This notebook asks what the model’s scores would mean for an actual lending decision. It turns the final model’s holdout predictions into default rates by risk group, the effect of flagging the riskiest applicants, and a comparison with a simple “no credit history, no loan” rule. It follows the success definition in the business problem statement: a lower default rate at the same approval rate, or a higher approval rate at the same default rate.

Data and model. Everything here uses the 61,502 holdout applicants the model never saw in training, so nothing is in-sample. The model is a refit of the final XGBoost model from `modeling.qmd` with the same hyperparameters, built by `rebuild_presentation_cache.R`. Its holdout AUC is 0.7550, against 0.7556 in the original run; the small gap comes from package-version differences.

How to read the results. Every policy below is a *retrospective* comparison on historical applicants whose outcomes are already known. These are illustrations of what the ranking could support, not measured results of a deployed policy. No costs, revenue, or loan pricing are included.

```
ho <- tibble(
  default    = as.integer(holdout_data$TARGET == 1),
  prob       = holdout_prob,
  credit     = holdout_data$AMT_CREDIT,
  has_bureau = holdout_data$has_bureau_record
)
n_ho      <- nrow(ho)
base_rate <- mean(ho$default)
per_10k   <- 10000 / n_ho

tibble(Item  = c("Holdout applicants", "Defaults", "Default rate", "Holdout AUC"),
       Value = c(num(n_ho), num(sum(ho$default)), pct(base_rate, 2),
                 sprintf("%.4f", check_auc))) |>
  knitr::kable()
```

Item	Value
Holdout applicants	61,502
Defaults	4,965
Default rate	8.07%
Holdout AUC	0.7550

1. Default rate by model risk decile

Applicants are ranked by predicted risk and split into ten equal groups (deciles). If the model ranks well, the default rate should rise steadily from decile 1 (safest) to decile 10 (riskiest).

```
ho <- ho |> mutate(decile = ntile(prob, 10))   # 10 = riskiest

deciles <- ho |>
  group_by(decile) |>
  summarise(applicants   = n(),
            avg_score    = mean(prob),
            default_rate = mean(default),
            defaults     = sum(default),
            .groups = "drop") |>
  arrange(desc(decile)) |>
  mutate(share_of_defaults = defaults / sum(defaults),
         cumulative_share  = cumsum(share_of_defaults))

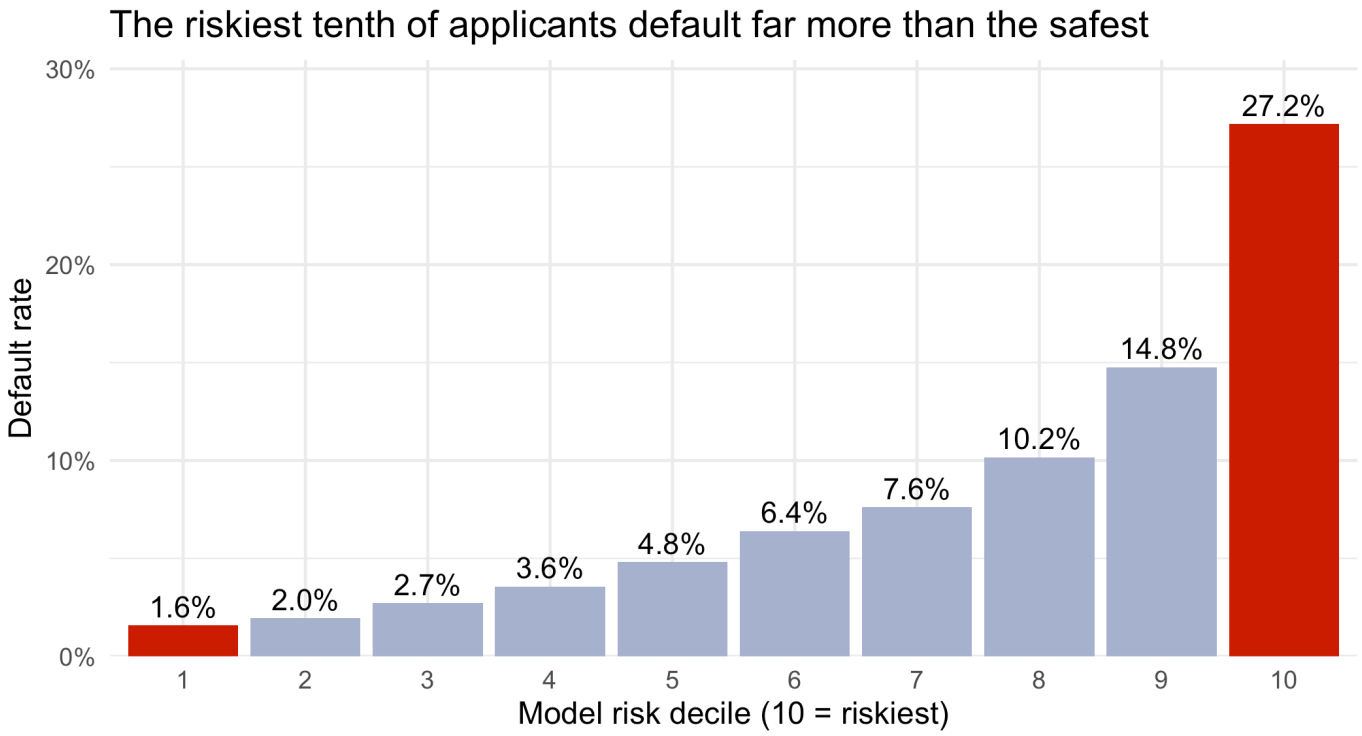
deciles |>
  mutate(across(c(avg_score, default_rate, share_of_defaults, cumulative_share),
                ~ pct(.x))) |>
  knitr::kable(caption = "Holdout applicants by model risk decile (10 = riskiest)")
```

Holdout applicants by model risk decile (10 = riskiest)

decile	applicants	avg_score	default_rate	defaults	share_of_defaults	cumulative_share
10	6150	25.5%	27.2%	1672	33.7%	33.7%
9	6150	14.6%	14.8%	908	18.3%	52.0%
8	6150	10.2%	10.2%	625	12.6%	64.6%
7	6150	7.7%	7.6%	468	9.4%	74.0%
6	6150	6.0%	6.4%	393	7.9%	81.9%
5	6150	4.8%	4.8%	296	6.0%	87.9%
4	6150	3.9%	3.6%	219	4.4%	92.3%
3	6150	3.2%	2.7%	167	3.4%	95.6%
2	6151	2.5%	2.0%	120	2.4%	98.0%
1	6151	1.7%	1.6%	97	2.0%	100.0%

```
write_csv(deciles, file.path(results_dir, "deciles.csv"))
```

```
decile_plot <- ggplot(deciles, aes(factor(decile), default_rate,
                                   fill = decile %in% c(1, 10))) +
  geom_col() +
  geom_text(aes(label = pct(default_rate)), vjust = -0.4, size = 6) +
  scale_fill_manual(values = c(`TRUE` = "#CC0000", `FALSE` = "#A9B4D0"), guide =
"none") +
  scale_y_continuous(labels = scales::percent, expand = expansion(mult = c(0, 0.12)))
+
  labs(title = "The riskiest tenth of applicants default far more than the safest",
       x = "Model risk decile (10 = riskiest)", y = "Default rate") +
  theme_minimal(base_size = 18)
decile_plot
```



Default rate by model risk decile on the 61,502 holdout applicants. The safest and riskiest deciles are highlighted.

```
ggsave(file.path(results_dir, "decile_chart.png"), decile_plot, width = 11, height =
6, dpi = 200)
```

Interpretation. The default rate rises at every step, from 1.6% in the safest decile to 27.2% in the riskiest, and the riskiest decile alone holds 33.7% of all defaults. A ranking this steep is what makes risk-based review possible: most of the risk sits in a small, identifiable group.

2. Flagging the riskiest applicants

Each row below flags the riskiest 5%, 10%, 15%, or 20% of holdout applicants and shows what happens to the default rate among everyone else. “Flagged” could mean declined, referred to an underwriter, or asked for more verification; the table does not assume which.

```
flag_policy <- function(q) {
  cutoff <- quantile(ho$prob, 1 - q)
  flag   <- ho$prob >= cutoff
  tibble(
    flagged_share         = mean(flag),
    defaults_caught       = sum(flag & ho$default == 1) / sum(ho$default),
    default_rate_flagged  = mean(ho$default[flag]),
    default_rate_approved = mean(ho$default[!flag]),
    relative_drop         = 1 - mean(ho$default[!flag]) / base_rate,
    good_borrowers_flagged = sum(flag & ho$default == 0) / sum(ho$default == 0),
    defaults_avoided_per_10k = sum(flag & ho$default == 1) * per_10k,
    defaulted_credit_avoided_per_10k = sum(ho$credit[flag & ho$default == 1]) *
per_10k,
    score_cutoff          = cutoff
  )
}
policies <- map_dfr(c(0.05, 0.10, 0.15, 0.20), flag_policy)

policies |>
  mutate(across(c(flagged_share:good_borrowers_flagged), ~ pct(.x)),
         across(c(defaults_avoided_per_10k, defaulted_credit_avoided_per_10k), num),
         score_cutoff = round(score_cutoff, 3)) |>
  knitr::kable(caption = "Flagging the riskiest X% of applicants")
```

Flagging the riskiest X% of applicants

flagged_share	defaults_caught	default_rate_flagged	default_rate_approved	relative_drop	good_borrowers_flagged	defaults_avoided_per_10k	defaulted_credit_avoided_per_10k	score_cutoff
5.0%	20.8%	33.6%	6.7%	16.6%	3.6%	168	82,149,624	0.235
10.0%	33.7%	27.2%	5.9%	26.3%	7.9%	272	134,638,620	0.179
15.0%	43.6%	23.5%	5.4%	33.6%	12.5%	352	176,575,492	0.144
20.0%	52.0%	21.0%	4.8%	40.0%	17.2%	419	213,317,236	0.120

```
write_csv(policies, file.path(results_dir, "policies.csv"))
```

Interpretation. Flagging more applicants catches more defaults but also flags more applicants who would have repaid (`good_borrowers_flagged`). Choosing where to draw the line is a business decision that needs Home Credit's own costs and risk appetite. This notebook shows the trade-off; it does not pick a cutoff.

3. Same approval rate: the model vs. "no credit history, no loan"

A traditional lender turns away anyone without a credit bureau record. This compares that rule with the model turning away the same number of applicants, namely the ones it rates riskiest.

Which applicants count as "no credit history". The rule here declines every applicant with **no credit bureau record** (`has_bureau_record == 0`), which is 14.3% of the training data. This is a broader group than the `thin_file` feature in `data_preparation.R` , which flags applicants with no bureau record *and* no credit

inquiry data. The `thin_file` feature is not used in this comparison.

What kind of comparison this is. It is retrospective: both policies are applied to historical holdout applicants whose outcomes are already known. The “no credit history, no loan” rule is a simplified benchmark, not Home Credit’s actual policy, and outcomes exist only for loans that were actually issued.

```
rule_decline  <- ho$has_bureau == 0
k             <- sum(rule_decline)
model_decline <- rank(-ho$prob, ties.method = "first") <= k

same_approval <- tibble(
  Policy = c("No credit history, no loan", "My model"),
  `Approval rate` = pct(c(1 - k / n_ho, 1 - k / n_ho)),
  `Default rate of approved loans` = pct(c(mean(ho$default[!rule_decline]),
                                           mean(ho$default[!model_decline])), 2),
  `Defaults per 10,000 applicants` = num(c(sum(ho$default[!rule_decline]),
                                           sum(ho$default[!model_decline])) *
per_10k),
  `No-bureau-record applicants approved` = pct(c(0, mean(!model_decline[ho$has_bureau
== 0])))
)
knitr::kable(same_approval)
```

Policy	Approval rate	Default rate of approved loans	Defaults per 10,000 applicants	No-bureau-record applicants approved
No credit history, no loan	85.7%	7.77%	666	0.0%
My model	85.7%	5.43%	465	80.1%

```
# Same default rate as the rule: how many applicants can the model approve?
# Note: this cutoff is found by searching the holdout's own outcomes for the
# largest approval count that keeps the default rate at or below the rule's, so
# the result is optimistic (see the note after the tables).
rule_rate  <- mean(ho$default[!rule_decline])
safe_first <- ho$default[order(ho$prob)]
approve_n  <- max(which(cumsum(safe_first) / seq_len(n_ho) <= rule_rate))
same_default <- tibble(
  `Default rate held at` = pct(rule_rate, 2),
  `Rule approves`  = pct(1 - k / n_ho),
  `Model approves` = pct(approve_n / n_ho),
  `Extra approvals per 10,000 applicants` = num((approve_n - (n_ho - k)) * per_10k)
)
knitr::kable(same_default)
```

Default rate held at	Rule approves	Model approves	Extra approvals per 10,000 applicants
7.77%	85.7%	99.3%	1,367

```
write_csv(same_approval, file.path(results_dir, "same_approval.csv"))
write_csv(same_default,  file.path(results_dir, "same_default.csv"))
```

Interpretation. At the same approval rate, the model-based policy approves loans with a lower default rate than the rule, and it still approves most of the applicants with no bureau record whom the rule turns away entirely. This is the more defensible of the two comparisons: the model only has to choose *which* applicants to decline, and the number declined is fixed by the rule.

Caution on the same-default-rate comparison. The second table should not be used as a headline result. Its cutoff was chosen *using the holdout outcomes*, by searching for the largest number of approvals that keeps the observed default rate at or below the rule's. A cutoff tuned on the same data it is evaluated on overstates how well it would hold up on new applicants.

4. Summary for the slides

```
p10 <- policies |> filter(abs(flagged_share - 0.10) < 0.005)
fewer <- (sum(ho$default[!rule_decline]) - sum(ho$default[!model_decline])) * per_10k

summary_lines <- c(
  sprintf("# Business impact results (holdout, %s applicants)", num(n_ho)),
  "",
  sprintf("- Holdout AUC check: %.4f", check_auc),
  sprintf("- Riskiest decile default rate: %s; safest decile: %s",
          pct(deciles$default_rate[deciles$decile == 10]),
          pct(deciles$default_rate[deciles$decile == 1])),
  sprintf("- Flag riskiest 10%%: catches %s of defaults; approved default rate %s ->
%s (%s lower)",
          pct(p10$defaults_caught), pct(base_rate, 2),
          pct(p10$default_rate_approved, 2), pct(p10$relative_drop)),
  sprintf("- Same approval rate as 'no credit history, no loan' (%s approved):
default rate %s (rule) vs %s (model); %s fewer defaults per 10,000 applicants",
          pct(1 - k / n_ho), pct(rule_rate, 2),
          pct(mean(ho$default[!model_decline]), 2), num(fewer)),
  sprintf("- Applicants with no credit bureau record the model approves at that rate:
%s",
          pct(mean(!model_decline[ho$has_bureau == 0]))),
  sprintf("- Same default rate as the rule (%s): model approves %s vs %s (%s more per
10,000 applicants); cutoff chosen using holdout outcomes, so optimistic",
          pct(rule_rate, 2), pct(approve_n / n_ho), pct(1 - k / n_ho),
          num((approve_n - (n_ho - k)) * per_10k)),
  "",
  "All comparisons are retrospective, on historical holdout applicants, with no costs
included."
)
writeLines(summary_lines, file.path(results_dir, "business_impact_summary.md"))
cat(summary_lines, sep = "\n")
```

```
## # Business impact results (holdout, 61,502 applicants)
##
## - Holdout AUC check: 0.7550
## - Riskiest decile default rate: 27.2%; safest decile: 1.6%
## - Flag riskiest 10%: catches 33.7% of defaults; approved default rate 8.07% ->
5.95% (26.3% lower)
## - Same approval rate as 'no credit history, no loan' (85.7% approved): default
rate 7.77% (rule) vs 5.43% (model); 200 fewer defaults per 10,000 applicants
## - Applicants with no credit bureau record the model approves at that rate: 80.1%
## - Same default rate as the rule (7.77%): model approves 99.3% vs 85.7% (1,367 more
per 10,000 applicants); cutoff chosen using holdout outcomes, so optimistic
##
## All comparisons are retrospective, on historical holdout applicants, with no costs
included.
```

5. Limitations

- **Retrospective, not prospective.** Every policy is applied to historical applicants with known outcomes. A deployed policy would face new applicants and changing conditions.
- **Issued loans only.** Outcomes exist only for loans Home Credit actually issued, so this analysis cannot show how previously rejected applicants would have performed.
- **No costs.** Defaults avoided are counted, not valued. Lost revenue from declined good borrowers, loan sizes, and pricing are not modeled, so no financial savings are claimed.
- **Fairness not assessed here.** Age and gender are among the model's influential predictors (see `modeling.qmd` and `02_personas.Rmd`). The project's model card (`model_card.qmd`) assesses fairness separately, at its own 0.25 threshold. It finds approval gaps by gender and education, and direct use of gender, marital status, and age that would block real-world use.
- **Association, not causation.** The model ranks applicants by characteristics associated with default; it does not identify causes.